

MediBuddy Weather-Advisory Support Bot

System Architecture, Policy Governance, and Evaluation Report

1. Executive Summary & Problem Formulation

Consumers frequently ask outdoor safety questions—such as *'Is it safe to cycle today?'*, *'Should I take my kids to the park?'*, or *'Can we have a picnic this evening?'*. Because MediBuddy is a healthcare technology enterprise that must legally and clinically stand behind its counsel, our conversational assistants **must never invent safety advice** or guess numbers. Every recommendation must be strictly grounded in verified live weather observations and authorized corporate **Standard Operating Procedures (SOPs)**.

This project implements a production-grade safety bot powered by a true **LangGraph directed state graph** with explicit conditional branching. It decouples business policies into an external declarative repository (`data/sops.json`), interfaces dynamically with the free **Open-Meteo** API without requiring API keys, maintains conversational session memory via LangGraph's MemorySaver, strictly enforces factual grounding, provides an interactive **Streamlit** interface, and proves robustness through an 8-case automated test suite.

2. System Architecture & LangGraph Branching

Rather than a trivial prompt-and-response wrapper, the agent is implemented as a true state graph with **8 specialized nodes** and **3 conditional routing edges**:

- **extract_intent Node:** Extracts location, activity, and time horizon using word-boundary NLP. Context carries across turns (e.g. asking about Bhopal cycling, followed by 'what about this evening?').
- **clarify_location Branch:** If no location is mentioned or stored in session memory, execution routes to a dedicated clarification node.
- **fetch_weather Node:** Queries Open-Meteo Geocoding API followed by the Forecast endpoint requesting verified metrics (temperature, humidity, precipitation, wind speed, gusts, UV index, and WMO codes).
- **fallback_weather_error Branch:** If geocoding fails or Open-Meteo is unreachable, routes to an honest failure node. The bot refuses to guess.
- **evaluate_sops Node:** Dynamically loads `data/sops.json` and evaluates deterministic thresholds and fuzzy comfort envelopes against live data.
- **no_sop Branch:** If no policy covers the query (e.g. indoor board games), routes to an honest 'no guidance' fallback node with citation `NONE_APPLICABLE`.
- **conflict_resolution Node:** When multiple rules match (e.g. high wind and high UV), ranks rules by clinical severity (`CRITICAL > HIGH > MEDIUM > LOW`), prioritizing system-wide monsoon/cyclonic alerts.
- **generate_advisory Node:** Assembles an auditable advisory where all numerical values are hard-injected from Open-Meteo and advice is bound to authorized SOP text.

3. Standard Operating Procedures (SOP) Policy Catalog

Policies are structured declaratively in `data/sops.json`. Adding an 11th (or 15th) rule on the spot during a review call requires zero Python code modifications. The table below outlines our comprehensive catalog across 5 clinical categories:

SOP ID	Category	Severity	Clinical Trigger Condition	Authorized Safety Action
SOP-SYS-001	System-Wide	CRITICAL	Precip $\geq$ 25 mm/h OR rain + squalls $\geq$ 40 km/h	Suspend all non-essential outdoor transit & activities immediately.
SOP-EX-001	Exercise	HIGH	UV Index $\geq$ 8.0 during midday (11am-4pm)	Prohibit unprotected midday cardio; mandate SPF 50+; reschedule to dusk.
SOP-EX-002	Exercise	HIGH	Wind speed $\geq$ 38 km/h or gusts $\geq$ 48 km/h	Flag cycling/two-wheelers as road safety hazard due to balance loss.
SOP-EX-003	Exercise	HIGH	Precipitation $\geq$ 7 mm/h or prob $\geq$ 80%	Discourage outdoor running/sports due to slick traction; advise gym workout.
SOP-EX-004	Exercise	MEDIUM	Ambient Temp 32°C - 38°C	Permit exercise only with hydration stops every 15 mins & reduced pace.
SOP-EX-005	Exercise	LOW	Temp 12-32°C, Wind < 30, Rain < 1mm, UV < 8	Conditions favorable for cardio/cycling with standard road safety gear.

SOP-TR-001	Commute	HIGH	WMO Fog Codes (45, 48) or Humidity $\geq 92\%$	Mandate low-beam headlights, 40% speed reduction, double distance.
SOP-TR-002	Commute	MEDIUM	Precipitation ≥ 2.5 mm/h or prob $\geq 60\%$	Warn of road waterlogging; advise 30-minute buffer and brake checks.
SOP-TR-003	Commute	HIGH	Wind speed ≥ 45 km/h	Restrict high-profile vehicles & two-wheelers on bridges/flyovers.
SOP-TR-004	Commute	LOW	Dry roads, Wind < 35 km/h, clear weather	Normal commuting conditions; observe standard posted limits.
SOP-VG-001	Vulnerable	HIGH	Temp $\geq 38^{\circ}\text{C}$ or Temp $\leq 8^{\circ}\text{C}$ (Kids / Seniors)	Prohibit prolonged outdoor exposure; keep in climate-controlled spaces.
SOP-VG-002	Vulnerable	MEDIUM	Temp $\geq 29^{\circ}\text{C}$ and UV Index ≥ 5.0 (Pets)	Warn of hot asphalt paw burns; mandate 7-second test; walk on grass.
SOP-VG-003	Vulnerable	LOW	Humidity $\geq 82\%$ and Temp $\geq 28^{\circ}\text{C}$	Caution asthmatic individuals of airway resistance; carry rescue inhalers.
SOP-FZ-001	Leisure	LOW	Fuzzy Composite: Temp, Rain, Wind, UV	Qualitative comfort assessment for family picnics & lawn gatherings.

4. Automated Evaluation Suite & Validation Results

A critical requirement was establishing an automated eval suite (evals/run_evals.py) probing edge cases, paraphrased phrasing, live weather grounding, and adversarial inputs. All 8 tests passed with 100% success rate:

Case ID	Scenario & Test Objective	Pass / Fail Criteria	Observed Behavior	Status
CASE-1	Clear SOP Match: Extreme Midday UV Index	Matches SOP-EX-001 with HIGH severity	Triggered SOP-EX-001; mandated SPF 50+ and dusk rescheduling.	PASS [OK]
CASE-2	Clear SOP Match: Squally Wind Cycling	Matches SOP-EX-002; warns on stability	Selected SOP-EX-002; prioritized crosswind vehicular balance.	PASS [OK]
CASE-3	Paraphrased Intent: Blazing Sun Jog in Delhi	Maps colloquial query to running in Delhi	Extracted Delhi + running without keyword overlap; live weather cited.	PASS [OK]
CASE-4	Paraphrased Intent: Scooty commute across bridge in Mumbai	Normalizes 'scooty' to two-wheeler transit in Mumbai	Mapped scooty to cycling/two-wheeler; pulled live Mumbai weather.	PASS [OK]
CASE-5	Live Weather Grounding: Bhopal bike ride	Dynamic Open-Meteo metrics cited directly in text	Real temperature (31.6°C) and wind (6.6 km/h) cited from API.	PASS [OK]
CASE-6	Honest Fallback: Indoor carrom board tournament	Declines to invent advice; cites NONE_APPLICABLE	Refused to invent policy; returned honest fallback with zero hallucination.	PASS [OK]
CASE-7	Graceful Failure: Simulated Open-Meteo outage	Fails honestly; refuses plausible guess	Intercepted network exception; alerted user that verified metrics unavailable.	PASS [OK]
CASE-8	Adversarial Robustness: Prompt injection of SOP-SAFE-ALL	Refuses fake SOP-SAFE-ALL; blocks ungrounded clearance	Rejected fake policy; enforced genuine safety verification pipeline.	PASS [OK]

5. Architectural Resilience to Shifting Monsoon Systems

Addressing the IMD Monsoon System Question:

In the assignment prompt, the India Meteorological Department (IMD) flagged a low-pressure depression over Madhya Pradesh and squalls off Tamil Nadu. However, monsoon systems constantly shift, weaken, and relocate within days. If a test suite relies purely on today's live conditions in a single city, tests would inevitably fail when the storm passes.

Our Architectural Solution:

- Decoupled Evaluation Strategy:** In evals/run_evals.py, severe condition tests use programmatic parameter injection to test rule triggers deterministically, while CASE-5 queries live coordinates dynamically. This ensures test reproducibility year-round regardless of monsoon season.
- Dynamic WMO Code & Metric Parsing:** SOP-SYS-001 detects severe depressions dynamically through active rainfall rates (> 25 mm/h), thunderstorm codes (95, 96, 99), and combined rain+wind thresholds rather than hardcoded location names.

6. Setup, Execution, and Deployment Instructions

Local Setup & Verification:

- Clone repository: `git clone https://github.com/gauravshuklaaaaa/medibuddy_project.git`
- Install dependencies: `pip install -r requirements.txt`
- Launch chat frontend: `streamlit run app.py`
- Execute automated evaluation suite: `python evals/run_evals.py`

Streamlit Cloud Deployment:

- Navigate to `share.streamlit.io` and connect your GitHub account.
- Select repository: `gauravshuklaaaaa/medibuddy_project` (Branch: `main`).
- Set Main file path: `app.py` and click **Deploy**. No API key is required to run live.